Current Protocols in Bioinformatics

November 8, 2005

Dr. David Emmert
Dept of Molecular & Cellular Biology
Harvard University
16 Divinity Avenue
Cambridge, MA 02167

Tel: (617) 496-5667
Fax: (617) 496-1354
emmert@morgan.harvard.edu

RE: CPBI Unit 9.6 rough pages

1. Please read the rough pages and mark any changes right in the text.
2. If you have large inserts to add, please supply us with a disk and hard copy of the insert(s) and indicate where they should go.
3. Read and answer all the queries on the Author/Editor Query Sheet and return this sheet with your Rough Pages.
4. If you have not returned your signed contract yet, you must return it at this time.
5. **Dr. Emmert should forward all materials by Tuesday, November 15th via Fed Ex or UPS to:**

   Dr. Lincoln Stein
   Cold Spring Harbor Laboratory
   1 Bungtown Road, PO Box 100
   Cold Spring Harbor, NY 11724

   Tel: (516) 367-8380
   Fax: (516) 367-8389
   lstein@cshl.org

6. **Dr. Stein should forward all materials by Friday, November 18th via Fed Ex or UPS to:**

   Allen Ranz
   Current Protocols Editorial Office
   John Wiley & Sons, Inc., MS 8.02
   111 River St.
   Hoboken, NJ 07030-5774

   Tel: (201) 748-6278
   Fax: (201) 748-6207
   aranz@wiley.com

cc: Liz Miranker, Lincoln Stein, Allen Ranz

Author/Editor Queries

1. Generally (especially Basic Protocols 4 and 5): Note that, for typesetting reasons, we have moved the large blocks of computer input/output originally in the text of this unit into a series of figures (graphical presentation provides greater control of the formatting, which appears significant in some of this material). We have also composed brief legends for these figures; please edit these or replace them as you see fit.

2. Basic Protocol 2 Necessary Resources, Files: Correct that the sample file GAME.xml is part of the XORT download?

3. Basic Protocol 3, (a) step 1a: We did some editing on the basis of the actual menu items seen at the top of the NCBI Nucleotide results page. It seems as if one needs only to choose Text from the "Send to" menu on the results page, and the text file simply appears; there does not appear to be a "Send" button. Is this correct, or have we misinterpreted something?

   (b) step 3: The last two sentence (bracketed by question marks) are very difficult to understand. Please clarify. In the first sentence, it is particularly difficult to figure out what "it" means; in the second; "this case" requires some clarification.

4. Basic Protocol 5 Necessary Resources: Correct that the sample files are provided as part of the XORT download?

5. Support Protocol 1 title: We have changed this title to make it more descriptive of the protocol; if you prefer other wording, please add it.

6. Support Protocol 1, step 6: Correct that you "Choose Postgresql from the 'database' menu?

7. Figure 9.6.18: As noted above, we have created this figure from the block of Unix commands under step 4 in the protocol. There is a line in this text ("fbreleases.xml") which does not appear to be part of the dialog between the computer and the user, and which is somewhat out of context here. Please add an annotation to step 4 of Basic Protocol 5 explaining what this means.

8. Table 9.6.1: (a) We have arranged the necessary additional Perl modules that you mention in Basic Protocol 5 in the form of a table. Please check to be sure that all of this is correct?

   (b) We have composed a tentative title for the table; please feel free to change or replace as you deem appropriate.

(c) XML:: Parser::PerlSAX: Since you had the wording “as part of libxml-perl” in lieu of a URL for this program, we copied over the URL for libxml-perl here. Correct?

(c) Is there a .tar.gz file for XML::RegExp? Please replace question mark.

9. Critical Parameters and Troubleshooting, “File system affecting file recognition in Cygwin environment,” last sentence: What are “*rc files”?

Using Chado to Store Genome Annotation Data

Chado was originally developed to integrate the information resources in two independent *Drosophila* databases. Since then it has evolved into a powerful ontology-driven genome database schema, in response to feedback from end users and the bioinformatics community. It is an integral component of the NIH/USDA ARS-funded Generic Model Organism Database (GMOD) project, and now supplies the database infrastructure for numerous software packages both within and outside the GMOD project. This Chado-compatible packages include the Gbrowse Web-based genome annotation browser (Stein et al., 2002) and Apollo (Lewis et al., 2002; also see *UNIT 9.5*), a genome annotation viewer and editor.

These protocols describe how to use the Chado relational database schema to store genome annotation data, in both the Unix/Linux/Mac OS X (Basic Protocol 1) and Windows environments (Support Protocol 1). This includes installing Chado and XORT in the Unix/Linux environment (Basic Protocol 1), obtaining the Chado schema data definition language script (DDL) and initializing the database (Basic Protocol 2), importing genome annotation data into the database (Basic Protocols 2 and 3), running useful queries across the data (Basic Protocol 4), and, finally, exporting the data into different standard data formats (Basic Protocol 5). An additional protocol will guide the reader through the acquisition and loading of genome data in GenBank flat-file format into Chado (Basic Protocol 3). A schematic representation of the organizational relationship between the protocols and the data flow for Chado is shown in Figure 9.6.1.

These protocols are intended for biologists and computer scientists who have experience working with whole-genome data as well as a basic working knowledge of Perl and RDBMS principles. It is assumed that the user has access to a computer running a Unix-based or PC/Windows operating system with the PostgreSQL RDBMS installed, and has permissions to create databases. All of the examples presented here come from FlyBase, which is a public database of genetic and molecular data for *Drosophila*.

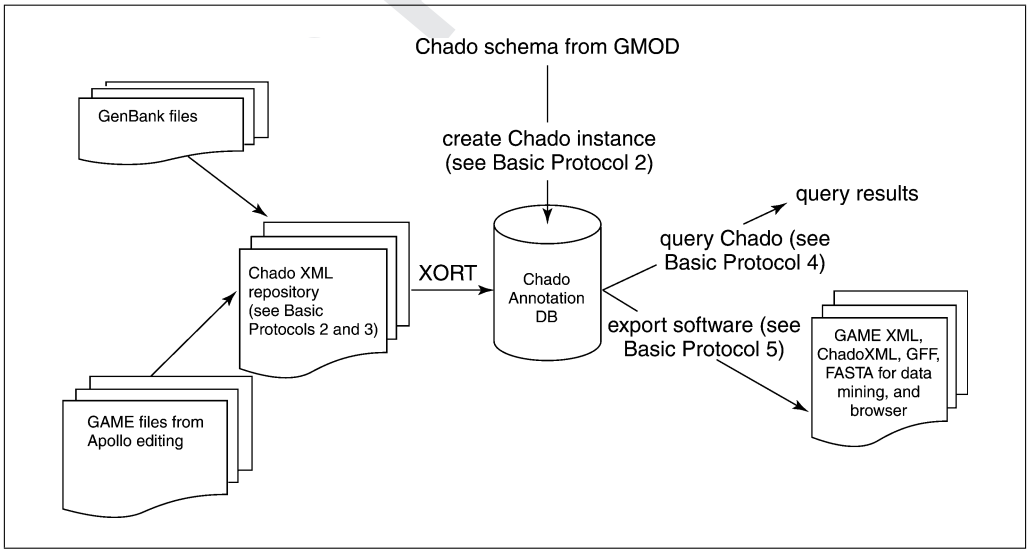

Figure 9.6.1 A schematic representation of the protocols and the organizational relationship between the protocols and data flow for Chado.

Contributed by Pinglei Zhou, David Emmert, and Peili Zhang
Current Protocols in Bioinformatics (2005) 9.6.1-9.6.27

INSTALLING CHADO AND XORT IN THE UNIX/LINUX ENVIRONMENT

This protocol describes the installation of Chado and XORT in the Unix/Linux environment and creation a new instance of a Chado database. Support Protocol 1 describes the corresponding procedures for a Windows computer.

Necessary Resources

Hardware

Any recent computer system running Macintosh OS X, Solaris, Linux, or other Unix variant

Software

Standard software:
PostgreSQL and BioPerl: Instructions for the installation of PostgreSQL and BioPerl under a Unix/Linux environment are beyond the scope of this unit; more information on these installation procedures can be found at *http://www.postgresql.org/docs/7.4/static/installation.html* (for PostgreSQL) and *http://www.bioperl.org* and *http://bioperl.org/Core/Latest/INSTALL* (for BioPerl)
The following software packages should be preinstalled:
Perl v. 5.0 or higher (available at *http://www.perl.org*)
PostgreSQL v. 7.3.2 or higher (available at *http://www.postgresql.org*)
Java 1.4.1 or higher (available at *http://java.sun.com*)
Nonstandard software:
These packages will need to be installed:
XML Parser (*http://search.cpan.org/~kmacleod/libxml-perl-0.08/*)
XML::DOM (*http://search.cpan.org/~tjmather/XML-DOM-1.43/*)
DBI (*http://search.cpan.org/~timb/DBI/*)
DBD-Pg (*http://search.cpan.org/dist/DBD-Pg/*)
GMODTools (*http://www.gmod.org*)
XML-XORT (available at *http://www.gmod.org*)
BioPerl (*http://www.bioperl.org/Core/Latest/index.shtml*)
For the purposes of this demonstration, it is assumed that the PostgreSQL database has been installed on `port=5432`, and that a user account with the username `zhou` and the password `zhoupg` has been created and has been granted "create database" privileges. It is also assumed that the XORT/Gamebridge package will be saved into the `/users/zhou/work` directory.

1. Download XORT. The XORT package can either be downloaded via SourceForge CVS or via FTP.
   a. To download via CVS, use the following command:
   ```
   $ cvs -z3 -d:pserver:anonymous@cvs.sourceforge.net:/
     cvsroot/gmod \co XML-XORT
   ```
   b. To download via FTP, use the following command:
   ```
   http://prdownloads.sourceforge.net/gmod/XML-XORT-
     0.001.tar.gz
   ```
2. Install XORT using the following commands:
   ```
   $ cd /users/zhou/work/XML-XORT
   $ perl Makefile.PL
   ```

3. `Makefile.PL` will then take the user through the configuration of XORT by asking a series of questions. In most cases, one can use the default configuration (presented in square brackets below).

```
What is the database name? chado
What is the database username? [zhou]
What is the password for 'zhou'? zhoupg
What is the database host? [localhost]
What is your database port? [5432]
Where will the tmp directory go?
  [/home/work/XML-XORT/tmp]
Where will the conf directory go?
  [/home/work/XML-XORT/conf]
Where is the DDL file?
  [/home/work/XML-XORT/conf/chado.ddl]
Where do you want to install XORT if other than
  default, press ENTER if default: [/tmp/xort]
$ make
$ make install
```

Besides installing the XORT executables, the steps above are required to collect database information that will be written to a very important configuration file called `chado.properties` *(default location:* `/home/work/XML-XORT/conf/chado.properties`*). The* `chado.properties` *file contains all the necessary information for XORT to connect to the Chado database that one is working with. If at some point it is necessary to connect to a different instance of Chado on the PostgreSQL server, one can either edit the* `chado.properties` *file to indicate the new Chado database name or create a new file, with the suffix* `.properties` *(e.g.,* `my_new_chado.properties`*). In order to be used by XORT, any* `.properties` *file needs to be located in the configuration directory (*`/home/work/XML-XORT/conf/`*). When executing XORT (see below), the* `-d` *parameter specifies which* `.properties` *file to use (e.g.,* `-d my_new_chado`*). For more documentation, see* `/home/work/XML-XORT/doc/readme_xort`.

BUILDING A CHADO ANNOTATION DATABASE

BASIC PROTOCOL 2

This protocol describes how to download the Chado DDL and use it to create an empty Chado database instance on a PostgreSQL server. It then describes how to convert a GAME XML file (the output format from Apollo) into ChadoXML, the XML format for all data going directly into and coming directly out of a Chado database when using XORT. Finally, the protocol describes how to use the XORT validator and loader utilities to validate ChadoXML and load it into the Chado database.

Necessary Resources

Hardware

Any recent computer system running Macintosh OS X, Solaris, Linux, or other Unix variant

Software

Standard software:

PostgreSQL and BioPerl: Instructions for the installation of PostgreSQL and BioPerl under a Unix/Linux environment are beyond the scope of this unit; more information on these installation procedures can be found at *http://www.postgresql.org/docs/7.4/static/installation.html* (for PostgreSQL) and *http://www.bioperl.org* and *http://bioperl.org/Core/Latest/INSTALL* (for BioPerl)

The following software packages should be preinstalled:
Perl v. 5.0 or higher (available at *http://www.perl.org*)
PostgreSQL v. 7.3.2 or higher (available at *http://www.postgresql.org*)
Java 1.4.1 or higher (available at *http://java.sun.com*)

Nonstandard software:
These packages will need to be installed:
XML Parser (*http://search.cpan.org/~kmacleod/libxml-perl-0.08/*)
XML::DOM (*http://search.cpan.org/~tjmather/XML-DOM-1.43/*)
DBI (*http://search.cpan.org/~timb/DBI/*)
DBD-Pg (*http://search.cpan.org/dist/DBD-Pg/*)
GMODTools (*http://www.gmod.org*)
XML-XORT (available at *http://www.gmod.org*)
BioPerl (*http://www.bioperl.org/Core/Latest/index.shtml*)

For the purposes of this demonstration, it is assumed that the PostgreSQL database has been installed on `port=5432`, and that a user account with the username `zhou` and the password `zhoupg` has been created and has been granted "create database" privileges. It is also assumed that the XORT/Gamebridge package will be saved into the `/users/zhou/work` directory.

Files

The sample data file `GAME.xml` used in this protocol is included as part of the XORT download

Create the Chado instance

1. Get the Chado schema DDL. The most recent Chado schema DDL is included with the XORT package distribution in directory `XML-XORT/conf/chado.ddl`. It may also be downloaded from the GMOD Sourceforge CVS using the following command:

```
$ cvs -z3 -d:pserver:anonymous@cvs.sourceforge.net:/
  cvsroot/gmod \co schema/chado
```

2. In psql client, create the Chado instance on the PostgreSQL database server using the Chado schema DDL:

```
psql> create database chado;
psql> \c chado;
psql> \i /users/zhou/work/XML-XORT/conf/chado.ddl
psql> \q
```

Load a GAME-XML File

3. Download GAME files from FlyBase. GAME XML files can be either created using Apollo (*UNIT 9.5*) or can be downloaded from the FlyBase Web site (which currently hosts annotation for two *Drosophila* species GAME XML) using the following command:

```
ftp://flybase.net/genomes/Drosophila_melanogaster/
  current/xml-game/
```

An example of GAME XML file format is shown in Figure 9.6.2.

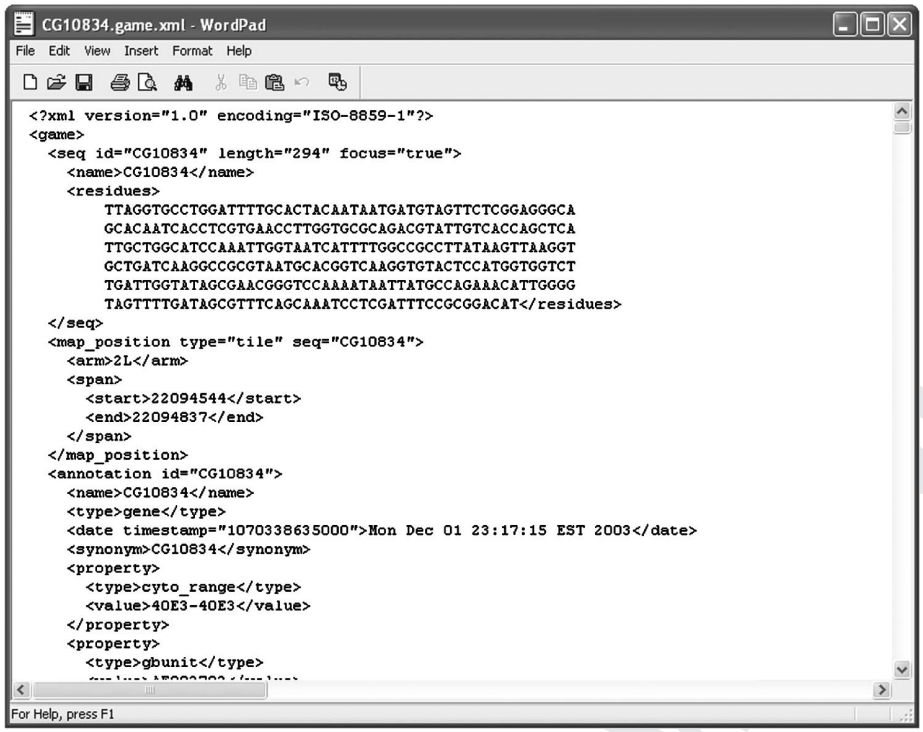

Figure 9.6.2 GAME XML format, which is one of the input formats for the annotation editor Apollo.

Convert GAME XML file into ChadoXML

The following steps use the sample file `GAME.xml`.

4. Convert GAME XML file into ChadoXML using the GTC converter in XORT. Software for converting between GAME XML and ChadoXML formats is included as part of the XML-XORT package. The following command will convert the sample GAME XML (`GAME.xml`) into ChadoXML (`chado.xml`):

```
$ java -cp /users/zhou/work/XML-XORT/conf/
  GAMEChadoConv.jar GTC GAME.xml chado.xml -a
```

   Examples of GAME XML and ChadoXML file formats are shown in Figures 9.6.2 and 9.6.3, respectively.

   Possible switches for GTC are:

   `-a`*: Convert all, both annotation and computational data.*
   `-g`*: Convert annotation data only.*
   `-c`*: Convert computational data only.*

Validate ChadoXML

5. Validate ChadoXML using the validator from the XORT package. The XORT validator serves several purposes. It may be used in stand-alone mode, which will verify the syntax of the ChadoXML, or it may be used in connection with the database, which will allow content testing to spot some potential problems before any update transactions are actually executed. The command syntax is:

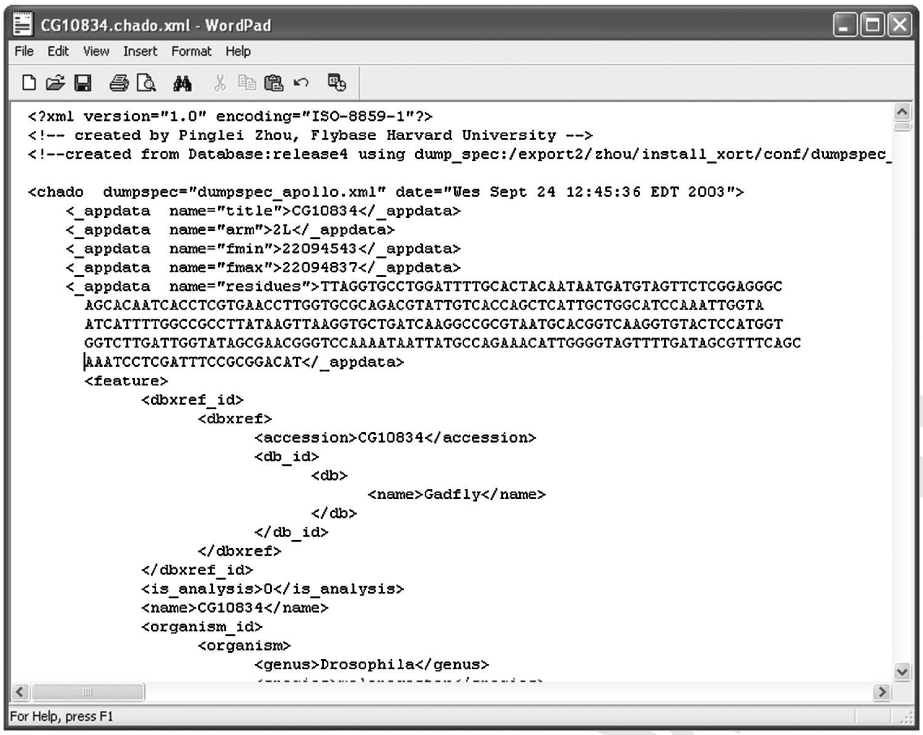

Figure 9.6.3 Structure for ChadoXML, which serves as intermediate format between Chado database and other file formats.

```
$ perl /users/zhou/work/XML-XORT/bin/
  xort_validator.pl -d chado -f chado.xml -v 1
```

Options and parameters include:

`-h`*: help message.*
`-d`*: database alias.*
`-f`*: ChadoXML file to be validated.*
`-b debug`*:* `0` *(default) without debug message,* `1` *with debug message.*
`-v 0` *or* `-v 1`*:* `0` *(default) without or* `1` *with database connection during validation process.*

In the example above, `-v 1` *is specified, causing XORT to connect to the database for content testing, which helps to identify data problems in the input ChadoXML before the loader actually attempting to load it into the database. Possible data problems originate from: (1) inconsistency between different version of the database schema, which may happen if the database schema has changed but converter configuration has not been followed, or (2) inconsistency in data implementation from different data sources, which may happen when ChadoXML generated by different projects and from different sources are loaded into the same Chado instance. The types of problems* `-v 1` *can detect include: (1) use of an object reference in ChadoXML before it has been defined, (2) attempting to delete an object which does not exist in the database, or (3) attempting to update an object without specifying enough constraints, which could lead to an erroneous update of multiple records.*

6. Load the file into the database using the loader from the XORT package. Assuming that the ChadoXML file validated successfully, the XORT loader can be used to update the database with the validated ChadoXML file. The command syntax is as follows:

```
$ perl /users/zhou/work/XML-XORT/bin/xort_loader.pl -d
  Chado -f chado.xml
```

The above command loads the ChadoXML into the database. At the end of loading, `xort_loader.pl` *indicates whether the loading was successful or not. If the loading was not successful, the loader attempts to indicate the nature of the problem it encountered.*

Options and parameters include:

`-h`*: help message.*
`-d`*: database alias.*
`-f`*: ChadoXML file to be loaded.*
`-b debug`*:* `0` *(default) without debug message,* `1` *with debug message.*

LOADING A GenBank FILE

BASIC PROTOCOL 3

Essentially all genome sequence and annotation data are submitted to GenBank by individual genome project teams and research groups. It is a common practice to acquire data by downloading relevant records from GenBank. A GenBank record can be fed into the BioPerl module Bio::SeqIO to build a Bio::Seq object. Peili Zhang at FlyBase Harvard has contributed a BioPerl module which converts all the sequence and annotation data contained in a Bio::Seq object into ChadoXML. The resulting ChadoXML can then be easily loaded into Chado using the XORT package. This protocol provides users an option to populate the Chado database with data from GenBank, a rich data source.

Necessary Resources

Hardware

Any recent computer system running Macintosh OS X, Solaris, Linux, or other Unix variant

Software

Standard software:
PostgreSQL and BioPerl: Instructions for the installation of PostgreSQL and BioPerl under a Unix/Linux environment are beyond the scope of this unit; more information on these installation procedures can be found at *http://www.postgresql.org/docs/7.4/static/installation.html* (for PostgreSQL) and *http://www.bioperl.org* and *http://bioperl.org/Core/Latest/INSTALL* (for BioPerl)
The following software packages should be preinstalled:
Perl v. 5.0 or higher (available at *http://www.perl.org*)
PostgreSQL v. 7.3.2 or higher (available at *http://www.postgresql.org*)
Java 1.4.1 or higher (available at *http://java.sun.com*)
Nonstandard software:
These packages will need to be installed:
XML Parser (*http://search.cpan.org/~kmacleod/libxml-perl-0.08/*)
XML::DOM (*http://search.cpan.org/~tjmather/XML-DOM-1.43/*)
DBI (*http://search.cpan.org/~timb/DBI/*)
DBD-Pg (*http://search.cpan.org/dist/DBD-Pg/*)
GMODTools (*http://www.gmod.org*)
XML-XORT (available at *http://www.gmod.org*)
BioPerl (*http://www.bioperl.org/Core/Latest/index.shtml*)
For the purposes of this demonstration, it is assumed that the PostgreSQL database has been installed on `port=5432`, and that a user account with the username `zhou` and the password `zhoupg` has been created and has been granted "create database" privileges. It is also assumed that the XORT/Gamebridge package will be saved into the `/users/zhou/work` directory.

Download GenBank flat files

There are two ways to download the GenBank records

1a. Go to the NCBI Web site (*http://www.ncbi.nlm.nih.gov*), choose Nucleotide from the Search drop-down menu, then enter the accession number or a valid identifier to search for the sequence. In the Search Results page, click on the relevant record to view its content. On the top of the record details page, choose the Text option from the "Send to" drop-down menu to save the page as a text file.

1b. Write a simple perl script making using of the BioPerl package (see sample code below) to download from GenBank over the Internet:

```
# Perl script to download AE003576 and save into file
  AE003576.g in local directory
use Bio::DB::GenBank;
use Bio::SeqIO;
my $gb = new Bio::DB::GenBank;
my $seq = $gb->get_Seq_by_acc('AE003576');
my $o = Bio::SeqIO->new(-file=>'>AE003576.gb',
  -format=>'genbank');
$o->write_seq($seq);
```

Convert GenBank flat files into ChadoXML and load into Chado

2. The GenBank data file may need some preprocessing on the feature coordinates before being converted into ChadoXML and loaded into the Chado database. In general GenBank records have record-based coordinates for all features in the record; for example, inside a GenBank mRNA record that is 2000 bp long, the gene and CDS features are both localized relative to this 2000-bp sequence, with the gene spanning 1..2000 and the CDS 35..1534. In a Chado database, however, it is far preferable to have a consistent coordinate system that is biologically meaningful and that allows better data management and data analysis. ?Therefore, it may have been chosen to localize sequence features on the chromosomes inside the particular Chado database at question. In this case, the chromosome that the GenBank record resides on must be determined by BLAST or by other means, the record-based coordinates in the GenBank record must be transformed into coordinates on the chromosome.?

3. After the necessary coordinate translation and/or other data preprocessing are completed, the GenBank flat data file can be converted into ChadoXML through a simple Perl script, making use of the aforementioned BioPerl module. An example of the code to use would be:

```
#Perl script to convert a GenBank formatted flat file
  to Chadoxml using BioPerl
use Bio::SeqIO;
#the GenBank records need to be pre-processed to
  convert the record-based coordinates
#to coordinates on the chosen coordinate system (e.g.,
  chromosome arm)
my $seqin = Bio::SeqIO->new(-format=>'genbank',
  -file=>'AE003576_processed.gb');
my $seq = $seqin->next_seq();
#AE003576 is a scaffold on Drosophila melanogaster
  chromosome arm 2L
my $arm = '2L';
```

```
my $seqout = Bio::SeqIO->new(-format=>'Chadoxml',
  -file=>'>AE003576.Chadoxml');
#if the optional input parameters 'src_feature' and
  'src_feat_type' are absent,
#the biological entity corresponding to the $seq object
  and its type are used as the
#src_feature and src_feat_type, respectively.
$seqout->write_seq(-seq=>$seq, -src_feature=>$arm,
  -src_feat_type=> 'chromosome_arm');
```

4. Load the ChadoXML into Chado as described in Basic Protocol 2.

QUERYING A CHADO ANNOTATION DATABASE USING SQL

BASIC PROTOCOL 4

This protocol presents some basic SQL queries for exploring annotation data in Chado. It is expected that once these queries, and the relational structures implicit in them, have been thoroughly understood, a user will be able to navigate through the tables storing annotation data in Chado and improvise queries as the need arises.

Necessary Resources

Hardware

Any recent computer system running Macintosh OS X, Solaris, Linux, or other Unix variant

Software

Standard software:

PostgreSQL and BioPerl: Instructions for the installation of PostgreSQL and BioPerl under a Unix/Linux environment are beyond the scope of this unit; more information on these installation procedures can be found at *http://www.postgresql.org/docs/7.4/static/installation.html* (for PostgreSQL) and *http://www.bioperl.org* and *http://bioperl.org/Core/Latest/INSTALL* (for BioPerl)

The following software packages should be preinstalled:
Perl v. 5.0 or higher (available at *http://www.perl.org*)
PostgreSQL v. 7.3.2 or higher (available at *http://www.postgresql.org*)
Java 1.4.1 or higher (available at *http://java.sun.com*)

Nonstandard software:

These packages will need to be installed:
XML Parser (*http://search.cpan.org/~kmacleod/libxml-perl-0.08/*)
XML::DOM (*http://search.cpan.org/~tjmather/XML-DOM-1.43/*)
DBI (*http://search.cpan.org/~timb/DBI/*)
DBD-Pg (*http://search.cpan.org/dist/DBD-Pg/*)
GMODTools (*http://www.gmod.org*)
XML-XORT (available at *http://www.gmod.org*)
BioPerl (*http://www.bioperl.org/Core/Latest/index.shtml*)

For the purposes of this demonstration, it is assumed that the PostgreSQL database has been installed on `port=5432`, and that a user account with the username `zhou` and the password `zhoupg` has been created and has been granted "create database" privileges. It is also assumed that the XORT/Gamebridge package will be saved into the `/users/zhou/work` directory.

```
select g.name as gn_name, g_c.fmin as gn_fmin, g_c.fmax as gn_fmax,
     g_c.strand as gn_strand, ch.name as gn_arm
from feature g, featureloc g_c, feature ch
where g.feature_id = g_c.feature_id   and g_c.srcfeature_id = ch.feature_id
     and g.name = 'oaf'
;
```

Figure 9.6.4 Example query to retrieve location information for the gene *oaf*.

```
--  gn_name | gn_fmin | gn_fmax | gn_strand | gn_arm
--  oaf     | 2492954 | 2498846 |         1 | 2L
-- (1 row)
```

Figure 9.6.5 Results returned for the query depicted in Figure 9.6.4.

```
select g.name as gn_name, tcv.name as feature_type,
     tx.name as tx_name, txc.fmin as tx_fmin, txc.fmax as tx_fmax,
     txc.strand as tx_strand, ch.name as tx_arm
from feature g, feature tx, feature_relationship g_tx, featureloc txc,
     feature ch, cvterm tcv
where      g.feature_id = g_tx.object_id
     and g_tx.subject_id = tx.feature_id
     and tx.feature_id = txc.feature_id
     and txc.srcfeature_id = ch.feature_id
     and tx.type_id = tcv.cvterm_id
     and tcv.name = 'mRNA'
     and g.name = 'oaf'
;
```

Figure 9.6.6 Example query to get transcripts and their locations for the gene *oaf*.

Using SQL to retrieve details of a single gene model

1. Design an SQL query (see Fig. 9.6.4 for example) to retrieve the location information for a given gene (e.g., *oaf*). This query joins three tables:
   a. `feature` (`g`, for the gene, *oaf*).
   b. `feature` (`ch`, for the chromosome arm).
   c. `featureloc` (`g_c`, localizing *oaf* on the chromosome arm).
2. The result of the query described in Figure 9.6.4 is shown in Figure 9.6.5.
3. For a given gene (eg, *oaf*), design a query to get transcripts and their locations (see Fig. 9.6.6 for example). This query joins six tables:

```
--   gn_name  |  feature_type  |  tx_name  |  tx_fmin  |  tx_fmax  | tx_strand | tx_arm

--   oaf      | mRNA           | oaf-RD    | 2492954   | 2498846   |         1 | 2L

--   oaf      | mRNA           | oaf-RB    | 2492954   | 2498234   |         1 | 2L

--   oaf      | mRNA           | oaf-RA    | 2492954   | 2498846   |         1 | 2L

--   oaf      | mRNA           | oaf-RC    | 2492954   | 2497357   |         1 | 2L

-- (4 rows)
```

Figure 9.6.7 Results returned for the query depicted in Figure 9.6.6.

```
select tx.name as tx_name, ecv.name as f_type, tx_ex.rank as ex_rank,
      ex.name as ex_name, exc.fmin as ex_fmin, exc.fmax as ex_fmax,
      exc.strand as ex_strand, ch2.name as ex_arm
from feature tx, feature ex, feature_relationship tx_ex, featureloc exc,
      feature ch2, cvterm ecv
where      tx.feature_id = tx_ex.object_id
      and tx_ex.subject_id = ex.feature_id
      and ex.feature_id = exc.feature_id
      and exc.srcfeature_id = ch2.feature_id
      and ex.type_id = ecv.cvterm_id
      and ecv.name = 'exon'
      and tx.name = 'oaf-RB'
order by ex_rank
;
```

Figure 9.6.8 Example query to get exons and their locations for a given transcript, “oaf-RB.”.

a. `feature` (`g` for the gene *oaf*).
b. `feature` (`tx` for the transcripts linked to the gene).
c. `feature_relationship` (`g_tx`, links gene and transcripts).
d. `featureloc` (`txc`, localizing transcripts on the chromosome arm).
e. `feature` (`ch` for the chromosome arm).
f. `cvterm` (`tcv`, for specifying the type of record linked to the gene).

4. The result of the query described in Figure 9.6.6 is shown in Figure 9.6.7.

5. For a given transcript (e.g., “oaf-RB”), design a query to get exons and their locations (see Fig. 9.6.8 for example). This query joins six tables:

a. `feature` (`tx`, for the transcript “oaf-RB”).
b. `feature` (`ex`, for the exons linked to the transcript).
c. `feature_relationship` (`tx_ex`, links transcript and exons).
d. `featureloc` (`exc`, localizing exons on the chromosome arm).

```
--  tx_name | f_type | ex_rank | ex_name | ex_fmin | ex_fmax | ex_strand | ex_arm

--  oaf-RB  | exon   |       1 | oaf:1   | 2492954 | 2493296 |         1 | 2L

--  oaf-RB  | exon   |       2 | oaf:2   | 2495373 | 2495487 |         1 | 2L

--  oaf-RB  | exon   |       3 | oaf:3   | 2495569 | 2495888 |         1 | 2L

--  oaf-RB  | exon   |       4 | oaf:5   | 2496398 | 2498234 |         1 | 2L

-- (4 rows)
```

Figure 9.6.9 Results returned for the query depicted in Figure 9.6.8.

```
select distinct(ex.name) as ex_name, exc.fmin as ex_fmin,
     exc.fmax as ex_fmax, exc.strand as ex_strand, ch.name as ex_arm,
     gn.name as gn_name
from feature gn, feature ex, feature_relationship gn_tx,
     feature_relationship tx_ex, featureloc exc, feature ch, cvterm ecv
where      gn.feature_id = gn_tx.object_id
     and gn_tx.subject_id = tx_ex.object_id
     and tx_ex.subject_id = ex.feature_id
     and ex.feature_id = exc.feature_id
     and exc.srcfeature_id = ch.feature_id
     and ex.type_id = ecv.cvterm_id
     and ecv.name = 'exon'
     and gn.name = 'oaf'
order by exc.fmin, exc.fmax
;
```

Figure 9.6.10 Example query to get exons and their locations for the gene *oaf*.

e. `feature` (`ch`, for the chromosome arm).
f. `cvterm` (`ecv`, for specifying the type of record linked to transcript).

The same query can be used to get the protein product and location, simply by switching `protein` *for* `exon` *in the text of Figure 9.6.8).*

6. The result of the query described in Figure 9.6.8 is shown in Figure 9.6.9.
7. For a given gene (eg, *oaf*), design a query to list exons and their locations (see Fig. 9.6.10 for example). This query joins seven tables:

a. `feature` (`gn`, for the gene, *oaf*).
b. `feature` (`ex`, for the exons linked to the gene via the transcripts).
c. `feature_relationship` (`gn_tx`, linking the gene and its transcripts).
d. `feature_relationship` (`tx_ex`, linking the transcripts and their exons).
e. `featureloc` (`exc`, localizing the exons on the chromosome arm).

```
--  ex_name | ex_fmin | ex_fmax | ex_strand | ex_arm | gn_name
--  oaf:1   | 2492954 | 2493296 |         1 | 2L     | oaf
--  oaf:2   | 2495373 | 2495487 |         1 | 2L     | oaf
--  oaf:3   | 2495569 | 2495888 |         1 | 2L     | oaf
--  oaf:6   | 2496398 | 2497357 |         1 | 2L     | oaf
--  oaf:5   | 2496398 | 2498234 |         1 | 2L     | oaf
--  oaf:4   | 2496398 | 2498846 |         1 | 2L     | oaf
-- (6 rows)
```

Figure 9.6.11 Results returned for the query depicted in Figure 9.6.10.

```
select distinct(a.program), a.sourcename
from feature hsp, feature ch, featureloc hsp_ch, featureloc hsp_al,
     analysisfeature af, analysis a
where hsp.is_analysis = 't'
     and hsp.feature_id = af.feature_id
     and af.analysis_id = a.analysis_id
     and hsp.feature_id = hsp_al.feature_id
     and hsp_al.feature_id = hsp_ch.feature_id
     and hsp_ch.srcfeature_id = ch.feature_id
     and ch.name = '2L'
     and hsp_ch.fmax < 50000
     and hsp_ch.fmin > 0
;
```

Figure 9.6.12 Example query to list types of analysis available and sets of data used in the analysis for a given genomic region (arm 2L, bases 1 to 49,999).

f. `feature` (`ch`, for the chromosome arm).

g. `cvterm` (`ecv`, for specifying the type of record linked to transcripts).

Note that the exons for a given gene model may be used in multiple transcripts, and, therefore, in order to return each exon only once (though they are related to the gene in Chado via the transcripts), it is necessary to formulate the query as "select distinct" (see Fig. 9.6.10).

8. The result of the query described in Figure 9.6.10 is shown in Figure 9.6.11.

SQL to retrieve supporting evidence tiers for specific gene model

9. For a given (genomic) region (e.g., arm 2L, bases 1 to 49,999), design a query to list types of analysis available and sets of data used in the analysis (see Fig. 9.6.12 for example). This query uses six tables:

a. `feature` (`hsp`, for HSP hits).

b. `feature` (`ch`, for the chromosome arm; e.g., `2L`).

c. `featureloc` (`hsp_ch`, linking HSPs to the chromosome arm).

```
--            program             |          sourcename
--  blastx_masked                 | aa_SPTR.dmel
--  blastx_masked                 | aa_SPTR.insect
--  blastx_masked                 | aa_SPTR.othinv
--  blastx_masked                 | aa_SPTR.othvert
--  blastx_masked                 | aa_SPTR.plant
--  blastx_masked                 | aa_SPTR.primate
--  blastx_masked                 | aa_SPTR.rodent
--  blastx_masked                 | aa_SPTR.worm
--  blastx_masked                 | aa_SPTR.yeast
--  dmel_r3_to_dmel_r4_migration | dmel_r3_affy_oligos
--  genscan_masked                | dummy
--  promoter                      | dummy
--  repeatmasker                  | dummy
--  sim4                          | na_DGC.in_process.dros
--  sim4                          | na_dbEST.diff.dmel
--  sim4                          | na_dbEST.same.dmel
--  sim4                          | na_gadfly.dros.RELEASE2
--  sim4                          | na_gb.dmel
--  sim4                          | na_transcript.dmel.RELEASE31
--  sim4                          | na_transcript.dmel.RELEASE32
--  tblastx_masked                | na_dbEST.insect
--  tblastxwrap_masked            | na_baylorf1_scfchunk.dpse
--  tblastxwrap_masked            | na_scf_chunk_agambiae.fa
-- (23 rows)
```

Figure 9.6.13 Results returned for the query depicted in Figure 9.6.12.

```
select distinct(al.name)
from feature al, feature ch, featureloc hsp_ch, featureloc hsp_al
where al.is_analysis = 't'
      and al.feature_id = hsp_al.srcfeature_id
      and hsp_al.feature_id = hsp_ch.feature_id
      and hsp_ch.srcfeature_id = ch.feature_id
      and ch.name = '2L'
      and hsp_ch.fmin > 0
      and hsp_ch.fmax < 50000
;
```

Figure 9.6.14 Example query to list aligned objects for a given genomic region (arm 2L, bases 1 to 49,999).

```
-- name
-- AT04953.5prime
-- AY051654
-- BU038928
-- CG11023-RA.31
-- CG11023-RA.32
-- ...
-- (178 rows)
```

Figure 9.6.15 Results returned for the query depicted in Figure 9.6.14.

```
select al.name as source_object, hsp.uniquename as hsp_name, hsp_al.fmin as
hsp_source_fmin, hsp_al.fmax as hsp_source_fmax, ch.name as target_object,
hsp_ch.fmin as hsp_target_fmin, hsp_ch.fmax as hsp_target_fmax, hsp_ch.strand as
hsp_target_strand
from feature al, featureloc hsp_al, feature hsp, featureloc hsp_ch, feature ch
where hsp_al.srcfeature_id = al.feature_id
      and hsp_al.feature_id = hsp.feature_id
      and hsp_al.rank = 1
      and hsp.feature_id = hsp_ch.feature_id
      and hsp_ch.rank = 0
      and hsp_ch.srcfeature_id = ch.feature_id
      and al.uniquename = 'AY129461'
order by hsp_source_fmin, hsp_source_fmax
;
```

Figure 9.6.16 Example query to retrieve the alignment details for the alignment of a given sequence against the chromosome arm (e.g., GenBank record “AY129461”).

d. `featureloc` (`hsp_al`, linking HSPs to the aligned object).
e. `analysisfeature` (`af`, linking HSP to analysis details).
f. `analysis` (`a`, for analysis details).

10. The result of the query described in Figure 9.6.12 is shown in Figure 9.6.13.
11. For a given (genomic) region (eg, arm 2L, bases 1 to 49,999), design a query to list aligned objects (see Figure 9.6.14 for example). This query uses four tables:

```
--  source_object | hsp_name | hsp_source_fmin | hsp_source_fmax | target_object | hsp_target_fmin
| hsp_target_fmax | hsp_target_strand
--  AY129461      | :5031336 |               0 |              54 | 2L            |           59189
|           59242 |               -1
--  AY129461      | :5031337 |              54 |             156 | 2L            |           58080
|           58182 |               -1
--  AY129461      | :5031338 |             156 |             713 | 2L            |           39300
|           39857 |               -1
--  AY129461      | :5031339 |             713 |             910 | 2L            |           38534
|           38731 |               -1
--  AY129461      | :5031340 |             910 |            1403 | 2L            |           34719
|           35212 |               -1
--  AY129461      | :5031341 |            1403 |            1450 | 2L            |           34557
|           34604 |               -1
--  AY129461      | :5031342 |            1450 |            1894 | 2L            |           33844
|           34288 |               -1
--  AY129461      | :5031343 |            1894 |            1981 | 2L            |           28981
|           29068 |               -1
--  AY129461      | :5031344 |            1981 |            2175 | 2L            |           28732
|           28926 |               -1
--  AY129461      | :5031345 |            2175 |            2401 | 2L            |           28014
|           28240 |               -1
--  AY129461      | :5031346 |            2401 |            2839 | 2L            |           27052
|           27490 |               -1
--  AY129461      | :5031347 |            2839 |            3038 | 2L            |           26765
|           26964 |               -1
--  AY129461      | :5031348 |            3038 |            4327 | 2L            |           25399
|           26688 |               -1
--  AY129461      | :5031349 |            4327 |            4341 | 2L            |           13464
|           13479 |               -1
-- (14 rows)
```

Figure 9.6.17 Results returned for the query depicted in Figure 9.6.16.

a. `feature` (`al`, for the aligned objects).
b. `feature` (`ch`, for the chromosome arm).
c. `featureloc` (`hsp_ch`, linking HSPs to the chromosome arm).
d. `featureloc` (`hsp_al`, linking HSPs to the aligned object).

Note that, since the aligned object is related to the chromosome via the HSP(s) that constitute the alignment, in order to avoid reporting a given aligned object multiple times (if there are multiple HSPs in its alignment to the arm), it is necessary to use the `distinct` *constraint.*

12. The result of the query described in Figure 9.6.14 is shown in Figure 9.6.15.

13. Query to retrieve the alignment details for the alignment of a given sequence against the chromosome arm (e.g., GenBank record "AY129461"; see Fig. 9.6.16). This query uses five tables:

    a. `feature` (`al`, for the aligned objects).
    b. `feature` (`hsp`, for the HSPs).
    c. `featureloc` (`hsp_ch`, localizing HSPs to the chromosome arm).
    d. `featureloc` (`hsp_al`, localizing HSPs to the aligned object).
    e. `feature` (`ch`, for the chromosome arm).

14. The result of the query described in Figure 9.6.16 is shown in Figure 9.6.17.

GENERATING STANDARD REPORTS FROM A CHADO ANNOTATION DATABASE

This protocol explains first how to dump data in ChadoXML format from a Chado annotation database using XORT, and then how to convert ChadoXML-formatted data files into the commonly used sequence annotation data formats GAME XML, GFF3, and FASTA,

Necessary Resources

Hardware

Any recent computer system running Macintosh OS X, Solaris, Linux, or other Unix variant

Software

Standard software:
PostgreSQL and BioPerl: Instructions for the installation of PostgreSQL and BioPerl under a Unix/Linux environment are beyond the scope of this unit; more information on these installation procedures can be found at *http://www.postgresql.org/docs/7.4/static/installation.html* (for PostgreSQL) and *http://www.bioperl.org* and *http://bioperl.org/Core/Latest/INSTALL* (for BioPerl)
The following software packages should be preinstalled:
Perl v. 5.0 or higher (available at *http://www.perl.org*)
PostgreSQL v. 7.3.2 or higher (available at *http://www.postgresql.org*)
Java 1.4.1 or higher (available at *http://java.sun.com*)
Nonstandard software:
These packages will need to be installed:
XML Parser (*http://search.cpan.org/~kmacleod/libxml-perl-0.08/*)
XML::DOM (*http://search.cpan.org/~tjmather/XML-DOM-1.43/*)
DBI (*http://search.cpan.org/~timb/DBI/*)
DBD-Pg (*http://search.cpan.org/dist/DBD-Pg/*)
GMODTools (*http://www.gmod.org*)
XML-XORT (available at *http://www.gmod.org*)
BioPerl (*http://www.bioperl.org/Core/Latest/index.shtml*)
For the purposes of this demonstration, it is assumed that the PostgreSQL database has been installed on `port=5432`, and that a user account with the username `zhou` and the password `zhoupg` has been created and has been granted "create database" privileges. It is also assumed that the XORT/Gamebridge package will be saved into `/users/zhou/work directory`.

Files

The sample data files `dumpspec_gene_model.xml` and `chado_dump.xml` used in this protocol are included as part of the XORT download

1. Export data in ChadoXML format using the following command:

```
$ perl /home/zhou/work/XML-XORT/conf/xort_dump.pl -d
  Chado -g /home/zhou/work/XML-XORT/conf/
  dumpspec_gene_model.xml -f output_file
```

Users can configure the XORT dumper to generate specific data sets into different ChadoXML formats using a project-specific "dump spec." The sample file (`dump-spec_gene_model.xml`*) is a typical dump spec to export Central Dogma structure into three-level XML file which, when converted to GAME XML, can be read by Apollo (see sample file:* `chado_dump.xml`*).*

```
<opt
  name="fbbulk-hetr3"
  relid="h3b2"     << key to fbreleases.xml entry
  ROOT="${GMOD_ROOT}/"
  TMP="${GMOD_ROOT}/tmp"
  datadir="genomes/Drosophila_melanogaster"    <<  this  will  be
created for output files,
                                                  from   ROOT  with
"$relfull" subfolder (see fbreleases.xml)
  >
...
  <include>fbreleases</include>
  <db    << most of these parameters are used, should match your
system values
    driver="Pg"
    name="BOGUS"  << superceeded by <release ... dbname=value >
    host="localhost"
    alt_host="bugbane.bio.indiana.edu"
    port="7302"
    user=""
    password=""
    />
...
```

fbreleases.xml

```
<release id="h3b2"   << matches main relid="h3b2"
    rel="hetr32b2"
    dbname="dmelhet_r32b2_Chado"   << PostgreSQL dbname
    relfull="dmel_hetr32b2_20050110"    <<  sub-folder  where  data
files go
    date="20050110"
    release_url="/annot/dmelhet-release3.2b2.html"
/>
```

Figure 9.6.18 Example of commands used to modify `conf/gmod.conf`, `conf/bulkfiles/fbreleases.xml`, and the main configuration file `conf/bulkfiles/fbbulk-hetr3.xml.fbbulk-hetr3.xml` to reflect the database.

```
:
     % cd /tmp/GMODTools/bin
     %       perl       -IGMODTools/lib       /bulkfiles.pl       -
conf../conf/bulkfiles/fbbulk-hetr3.xml -
dnadump -featdump
```

Figure 9.6.19 Example of commands used for generating report files.

2. Convert ChadoXML to GAME XML. Software for converting between GAME XML and ChadoXML formats is included as part of the XML-XORT package. The following command will convert the sample GAME XML (`GAME.xml`) into ChadoXML (`chado.xml`):

   ```
   $ java -cp /users/zhou/work/XML-XORT/conf/
     GAMEChadoConv.jar CTG chado.xml GAME.xml -a
   ```

   Possible switches for GTC are:

   `-a`*: Convert all, both annotation and computational data.*
   `-g`*: Convert annotation data only.*
   `-c`*: Convert computational data only.*

3. Produce GenBank flat-file format report. For the moment, the preferred way to produce GenBank flat-file format records from GAME XML is to load the GAME XML file into Apollo, and then save again in GenBank format. See *UNIT 9.5* for details on how to do this.

4. Produce bulk FASTA or GFF3 dumps. Assume that the GMODTools package has been downloaded from the GMOD Web site (see Necessary Resources) and saved in the `/tmp` directory. The following three configuration files need to be modified to reflect the database connection: `conf/gmod.conf`, `conf/bulkfiles/fbreleases.xml`, and the main configuration file `conf/bulkfiles/fbbulk-hetr3.xml.fbbulk-hetr3.xml` using a series of commands similar to those in Figure 9.6.18.

   The file name is not important; any file name with proper information can be used.

   Don Gilbert at FlyBase Indiana developed the necessary software to generate FASTA/GFF3 report files.

5. After editing the configuration files to reflect the project specific information, it is possible to generate report files using a command similar to the one in Figure 9.6.19. This prepares the Chado database for reporting, assuming configuration file `fbbulk-hetr3.xml` has the correct data information:

   ```
   % perl -IGMODTools/lib bulkfiles.pl -conf
     ../conf/bulkfiles/fbbulk-hetr3.xml -make
   ```

   makes all FASTA and GFF3 files for all chromosomes.

   ```
   % perl -IGMODTools/lib bulkfiles.pl -conf
     ../conf/bulkfiles/fbbulk-hetr3.xml -chr X -format
     fasta -make
   ```

   makes subset of FASTA files for chromosome X, (subset of GFF3 for chromosome X if replacing FASTA with GFF).

INSTALLING SOFTWARE FOR A UNIX-LIKE ENVIRONMENT ON A PC

For many reasons, much open-source bioinformatics software runs on Linux, Mac OS X, Solaris, or one of the many other Unix variants. Nevertheless, many biologists favor the Windows operating system, partially due to the user-friendly GUI software that can be used under this system. Cygwin is a Linux-like environment for Windows that attempts to solve the conflict between Unix and Windows operating systems. With Cygwin, one can run most programs with Linux-like command. This Support Protocol guides the user through the steps needed to create, populate, and use the Chado database in a Windows environment running Cygwin.

Necessary Resources

Hardware

PC with ≥256 Mb RAM, ≥1 GHz processor, and ≥10 Gb hard disk

Software

Cygwin (*http://www.cygwin.com*)
Perl v. 5.0 and higher (available at *http://www.perl.org*)
PostgreSQL v. 7.3.2 and higher (come with Cygwin installation)
Java 1.4.1 and higher (available at *http://java.sun.com*)
XML::DOM (available at *http://search.cpan.org/~tjmather/XML-DOM-1.43/*)
XML::Parser (available at *http://sea rch.cpan.org/~kmacleod/libxml-perl-0.08/*)
DBI (available at *http://search.cpan.org/~timb/DBI/*)
DBD-Pg (available at *http://search.cpan.org/dist/DBD-Pg/*)
XML-XORT (available at *http://www.gmod.org*)

1. Download and install the Windows version of Java 1.4.2. From *http://java.sun.com/j2se/1.4.2/download.html*, click "Download Windows J2SE SDK." When the download is completed, click on the downloaded executable file to start the installation. After installation, add the Java directory to `CLASSPATH`. See Troubleshooting section ("How to set environment variable in Windows environment") for more details.

   To test if set it properly, open a DOS window, type `java -h`*, and check if the command parameters are displayed. If not, check to see if the* `CLASSPATH` *variable is set properly. Remember the every time the variable value is modified, a new DOS window must be opened to refresh the setting.*

2. Download and install the latest version of Cygwin at *http://www.cygwin.com/* by clicking on one of the several Install Cygwin Now links on the home page. This will start an interactive download. After several steps, a screen will appear that makes it possible to select which packages to download (Fig. 9.6.20). The groups that must be installed in order to install the rest of the package are tabulated in Figure 9.6.21. Note that some of these will be installed by default, and some are interdependent, so that when one is chosen, another may automatically be selected.

   For demonstration purpose, it is assumed that the Root Install Directory is set as `d:\cygwin`*..*

3. Download and install Perl under Cygwin. As stated in the last step, if `interpreter->perl` and `interpreter->perl-libwin32` are chosen, the setup process automatically installs the core Perl module.

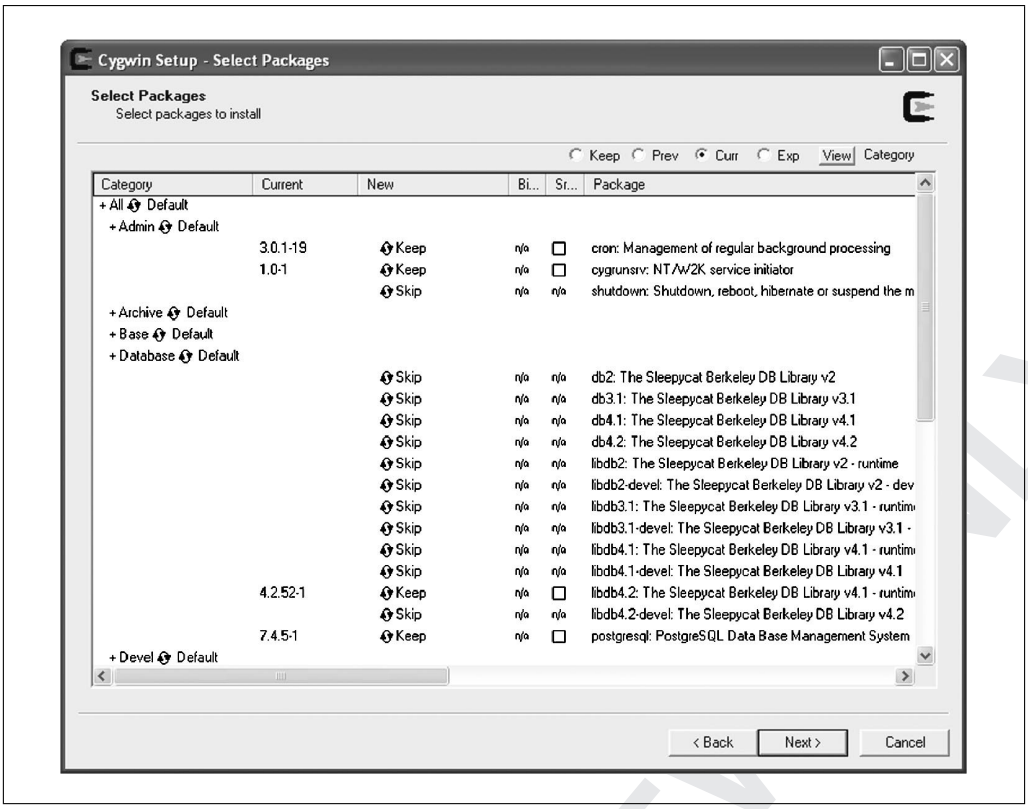

Figure 9.6.20 Screen shot of Cygwin setup process.

4. Download and install additional Perl modules. The modules listed in Table 9.6.1 are not installed with core Perl, and need to be downloaded and installed from CPAN. Remember to download and install in the correct order.

5. Here `bioperl-1.5.0.tar.gz` is used as an example to detail the installation. From *http://www.bioperl.org*, click Download, then download the core BioPerl. Assuming that `bioperl-1.5.0.tar.gz` has been saved in `d:/cygwin/tmp`, install using the following series of commands:

```
$ cd /tmp
$ gunzip bioperl-1.5.0.tar.gz
$ tar xvf bioperl-1.5.0.tar
$ cd bioperl-1.5.0
$ perl Makefile.PL
$ make
$ make test
$ make install
```

6. Download, install and configure PostgreSQL under Cygwin. The easiest way is to download and install PostgreSQL via Cygwin setup. Choose PostgreSQL from the "database" menu, then PostgreSQL will be downloaded and installed under Cygwin. Subsequently, configure PostgreSQL as follows:

   a. Set the environment variable CYGWIN to "server." Refer to troubleshooting section ("How to set environment variable in Window environment") for details.

   b. Install and configure cygserver using the command:

```
$ cygserver-config
```

```
admin->cron                                                    (BIN)
admin->cygrunsrv                                               (BIN)
base->ash                                                       (BIN)
base->bash                                                      (BIN)
base->cygwin                                                    (BIN)
base->diff                                                      (BIN)
base->diffutils                                                 (BIN)
base->fileutils                                                 (BIN)
base->findutils                                                 (BIN)
base->gawk                                                      (BIN)
base->gdbm                                                      (BIN)
base->grep                                                      (BIN)
base->gzip                                                      (BIN)
base->libncurses5                                               (BIN)
base->libncurses6                                               (BIN)
base->libreadline4                                              (BIN)
base->libreadline5                                              (BIN)
base->login                                                     (BIN)
base->ncurses                                                   (BIN)
base->readline                                                  (BIN)
base->sed                                                       (BIN)
base->sh-utils                                                  (BIN)
base->tar                                                       (BIN)
base->termcap                                                   (BIN)
base->terminfo                                                  (BIN)
base->textutils                                                 (BIN)
base->which                                                     (BIN)
base->zlib                                                      (BIN)

database->postgresql

Devel->cvs
Devel->cmake
Devel->cygipc
Devel->expat
Devel->gcc
Devel->gcc-core
Devel->make

Devel->cvs
Gnome->libxml2

Interpreters->expat
Interperters->libxml2
Interpreters->perl
Interpreters->perl-libwin32

Misc->setup

Shell->ash
Shell->bash
System->rebase
```

Figure 9.6.21 Groups that must be installed in order to install Cygwin.

Table 9.6.1 Perl Modules Needed for Running Chado Under Windows

Module	URL	Current version file
HTML::Tagset	*http://search.cpan.org/~sburke/HTML-Tagset-3.04/*	`HTML-Tagset-3.04.tar.gz`
HTML::TokeParser	*http://search.cpan.org/dist/HTML-Parser*	`HTML-parser-3.45.tar.gz`
URI	*http://search.cpan.org/dist/URI/*	`URI-1.35.tar.gz`
LWP::UserAgent	*http://search.cpan.org/dist/libwww-perl/*	`libwww-perl-5.803.tar.gz`
XML::RegExp	*http://search.cpan.org/~tjmather/XML-RegExp-0.03/*	`?`
XML::DOM	*http://search.cpan.org/~tjmather/XML-DOM-1.43/*	`XML-DOM-1.43.tar.gz`
XML::Parser	*http://search.cpan.org/~kmacleod/libxml-perl-0.08/*	`libxml-perl-0.08.tar.gz`
XML::Parser::PerlSAX	*http://search.cpan.org/~kmacleod/libxml-perl-0.08/*	`libxml-perl-0.008.tar.gz`
XML::Writer	*http://search.cpan.org/~josephw/XML-Writer-0.520/*	`XML-Writer-0.530.tar.gz`
DBI	*http://search.cpan.org/~timb/DBI/*	`DBI-1.47 DBI-1.47.tar.gz`
DBD::Pg	*http://search.cpan.org/dist/DBD-Pg/*	`DBD-Pg-1.32.tar.gz`
XML::Simple	*http://search.cpan.org/~grantm/XML-Simple-2.12/*	`XML-Simple-2.14.tar.gz`
BioPerl	*http://www.bioperl.org/Core/Latest/index.shtml*	`bioperl-1.5.0.tar.gz`

c. Start cygserver:
```
$ /usr/sbin/cygserver &
```
d. Initialize PostgreSQL:
```
$ initdb -D /var/postgresql/data
```
e. Start the PostgreSQL postmaster:
```
$ postmaster -D /var/postgresql/data -i &
```
f. Connect to PostgreSQL:
```
$ psql -d template1 -U username -W
```
Here, username/password are the same as the Window login username/password.

g. Note that, the next time one connects to the database, only the following steps are necessary:
```
$ postmaster -D /var/postgresql/data -i &
$ psql -d template1 -U username -W
```
h. To shut down the database properly before logging out use:
```
$ pg_ctl stop -D /var/postgresql/data -m fast
```

7. Download and install XORT as described in Basic Protocol 1.
8. Create the Chado instance as described in Basic Protocol 2.

COMMENTARY

Background Information

Controlled vocabularies and relationships between biological objects in Chado

According to Chado schema documentation, most objects with biological meanings are defined as "features" in Chado, which are localized (if they have been localized) to a specific location on a genomic contig, linked via the featureloc table. Feature relations (in the feature_relationship table) are used to establish relationships between features other than localizations. For example, the relationship between a gene and a transcript is represented by a feature relationship of type "partof," while an allele is linked to a gene by a feature relationship of the type "AlleleOf." The core representation of genes, transcripts, and proteins, and the relationships between them, is

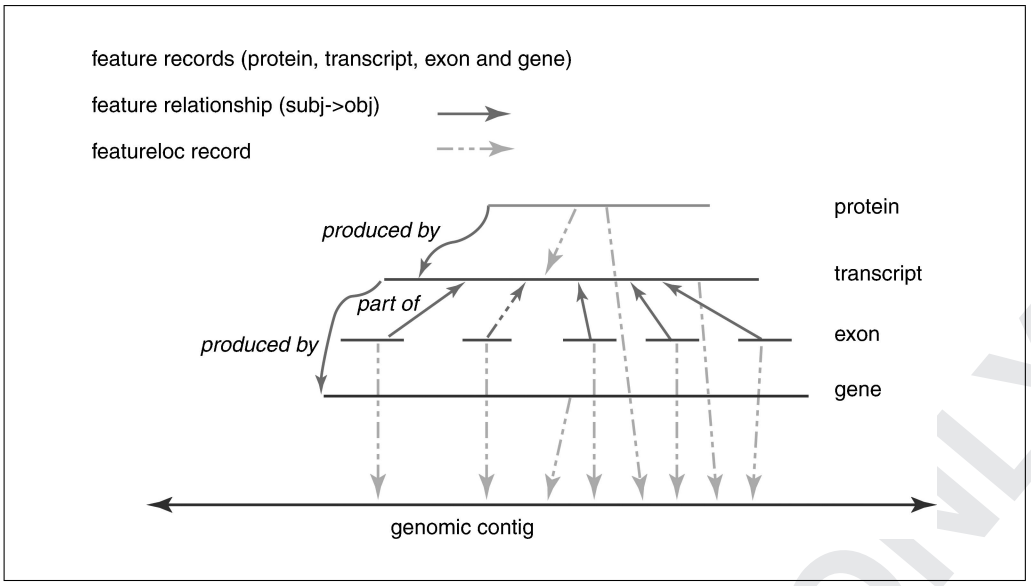

Figure 9.6.22 The Central Dogma model for a protein-coding gene with one known spliced transcript. The dashed lines denote the featureloc records of features aligned to the genomic contig, while the solid lines denote the feature_relationship records between two features (subject and object). For the color version of this figure go to *http://www.currentprotocols.com*.

referred to as the Central Dogma. In Chado, it is implemented as a three-level hierarchical structure: genes, transcripts, proteins, and exons are stored as features and the transcript features are stored as "partof" the gene feature through feature_relationship, whereas the exon and protein features are saved as "partof" and "producedby" the transcript features, respectively (Fig. 9.6.22).

Alignment data and other evidence in Chado

The evidence to support genome annotation includes gene prediction generated by programs such as GenScan (Burge and Karlin, 1997) and Genie (Reese et al., 2000), and other biological data such as ESTs aligned using the program Sim4 (Florea et al., 1998) and protein homologies revealed by BLASTX (Altschul et al., 1990; Mungall et al., 2002). As with the Central Dogma, they are stored in Chado as features with featureloc and feature_relationship information (Fig. 9.6.23).

Understanding the GAME XML format

GAME (genome annotation markup extensive) XML, designed by the Berkeley Drosophila Genome Project, is the major input file format for Apollo. The basic structure includes the following elements:

1. `<game>`: The root element, this represents the curation of one or more sequences of DNA, RNA, or amino acids. Most commonly, the `<game>` element represents the curation of a single sequence.

2. `<seq>`: Represents a sequence of DNA, RNA, or amino acids. There is generally one `<seq>` in the document representing the primary sequence being curated, and other `<seq>`'s that support the curation of the primary sequence. The primary `<seq>` is directly under the `<game>` element, and is identified by having its "focus" attribute set to "true." Each `<seq>` has one or more `<db_xref>`'s. `<db_xref>` indicates where the `<seq>` can be found using a particular unique identifier. For Drosophila curation, BDGP uses the primary `<seq>` to represent an accession, and other `<seq>`'s to represent cDNAs, protein coding sequences, and homologous sequences that are referenced by computational analyses such as tblastx.

3. `<annotation>` This represents a set of related sequence features and a collection of genetic information describing them. The term "sequence feature" means a segment of DNA. An annotation will generally contain a number of `<feature_set>`'s, each of which represents a set of related sequence features that have a specific location. A `<feature_set>` can contain nested `<feature_set>`'s (although in practice this has not yet occurred), as well as one or more `<feature_span>`'s, each of which represents an individual sequence feature. A `<feature_span>` can contain `<evidence>`, which specifies a result id and result type. An `<annotation>` can have one or more `<db_xref>`'s.

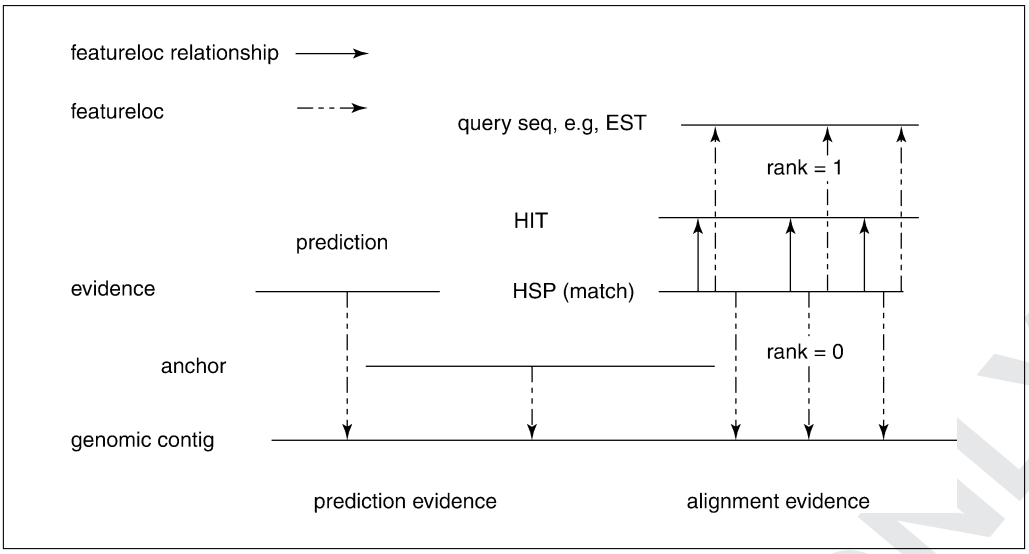

Figure 9.6.23 Data implementation of prediction and alignment evidence in Chado to support genome annotation. The dashed line denotes the featureloc of features aligned to genomic contig, while solid line denotes the feature relationship between two features.

For Drosophila curation, the types of annotations are: gene, pseudogene, transposon, tRNA, rRNA, snRNA, snoRNA, "misc. noncoding RNA," and "miscellaneous curator's observation." For an `<annotation>` of type "gene", one `<feature_set>` element represents each transcript, and for each transcript, one `<feature_span>` element represents each exon.

4. `<computational_analysis>` Contains evidence from computational analysis programs such sim4 and blastx. `<result_set>`'s and `<result_span>`'s represent a tree structure of results, with `<result_set>`'s representing branch nodes (e.g., gene matches), and `<result_span>`'s representing leaf nodes (e.g., exon matches). The elements `<feature_set>`, `<feature_span>`, `<result_set>`, and `<result_span>` run parallel to one another. They allow multiple levels of nesting and have physical location(s) on sequences. The key differences are that "features" have results as evidence and "results" have some form of an associated score for the assay. `<seq_relationship>`'s provides the locations on the underlying `<seq>`'s.

Critical Parameters and Troubleshooting

File system affecting file recognition in Cygwin environment

If possible, install Cygwin on a drive or partition that's NTFS-formatted instead of FAT32-formatted. When installing Cygwin on a FAT32 partition, it is not possible to set permissions and ownership correctly, which may be problematic in certain situations. If trying to use some application or resource "outside" of Cygwin and a problem is encountered, remember that Cygwin's path syntax may not be the correct one. Cygwin understands `/home/jacky` or `/cygdrive/e/cygwin/home/jacky` (when referring. e.g., to the `E:` drive), but the external resource may want `E:/cygwin/home/jacky`. Depending on these issues, the *rc files may end up with paths written in these different syntaxes.

File permission under Cygwin

Cygwin PostgreSQL may fail to start or not function properly if certain files and directories have incorrect permissions. The following usually solves such problems:

```
$ chmod a+rwx /tmp
$ chmod a+rx /usr/bin
  /usr/bin/*
$ chmod a+rw /var/log # could
  adversely affect other
  daemons
```

Make test for DBD::Pg under Cygwin

While installing DBD::Pg, the make tests are designed to connect to a live database. The following environment variables must be set for the tests to run:

```
DBI_DSN=dbi:Pg:dbname=
  <database>
DBI_USER=<username>
DBI_PASS=<password>
```

Under Cygwin, set those variables as follows: (assuming that one is logged in as `system administrator: zhou/zhoupgsql`):

```
$ DBI_DSN=dbi:Pg:dbname=
  template1
$ export DBI_DSN
$ DBI_USER=zhou
$ export DBI_USER
$ DBI_PASS=zhoupgsql
$ export DBI_PASS
```

Memory issue when converting big GAME XML files into ChadoXML

The converter uses DOM structure to build the tree structure into memory. It requires a large amount of memory. If the converter displays an "out of memory" error, it is possible to explicitly allocate more memory (e.g., 500 Mb if possible) to the Java process with the following command:

```
java -Xms500M ... ... ./conf/
  GAMEChadoConv.jar GTC ... ..
```

If the machine has limited memory, one way to get around this problem is to copy `GAMEChadoConv.jar` and input the GAME file to another machine with Java installed that has more memory, because this utility is independent of other modules.

How to enable PostgreSQL for TCP/IP

In order to use DBI to access PostgreSQL (which is required by XML-XORT), it is necessary to have one's database enabled for TCP/IP connections. Either start the database up with the `-i` switch:

```
$ postmaster -D /var/
  postgresql/data -i &
```

or enable TCP/IP settings in the database. Edit the `/var/postgresql/data/postgresql.conf` file and set:

```
tcpip_socket = true
```

How to set environment variables in the Windows environment

As an example, to add `d:\jdk1.4\bin` to `CLASSPATH` according to the instructions for the respective operating system as described under the headers below.

For Windows NT 4

Activate the Control Panel from the Start menu (under Settings), then double-click on the System icon. Choose the Environment tab. Select the `CLASSPATH` variable (in the System Variables section) by clicking on it. In the Value edit box, add `d:\jdk1.4\bin` to the front of the variable definition, but be sure not to overwrite what is already there. Note the semicolon, which separates path segments. Click the Set button, then OK.

For Windows 2000

Activate the Control Panel from the Start menu (under Settings), then double-click on the System icon. Select the Advanced tab and click on Environment Variables. Select the `CLASSPATH` variable (in the System Variables section), then click Edit. Add `d:\jdk1.4\bin` to the front of the variable definition, but be sure not to overwrite what is already there. Note the semicolon, which separates path segments. Click OK on each successive screen to return to the Control Panel, then close the Control Panel window.

For Windows XP Home or Professional

Activate the Control Panel from the Start menu (under Settings). From the vertical menu on the left, select Switch to Classic View, then click on the System icon. Select the Advanced tab and click on Environment Variables. Select the `CLASSPATH` variable (in the System Variables section), then click Edit. Add `d:\jdk1.4\bin` to the front of the variable definition, but be sure not to overwrite what is already there. Note the semicolon, which separates path segments. Click OK on each successive screen to return to the Control Panel, then close the Control Panel window.

For Windows 98SE

Activate the Run dialog box by selecting Run from the Start menu, then enter notepad `c:\autoexec.bat`. Click OK. When the editor window appears, add the following line to the end of the file:

```
set CLASSPATH=
  d:\jdk1.4\bin;%CLASSPATH%
```

Save the file, quit the editor, then restart the computer.

Note that, to add a forward-slash directory to an existing value, one should separate the new value from the old one by a colon (`:`) instead of a semicolon (`;`). For example to add `/home/XML-XORT/lib` to an existing PERL5LIB that already has the value `/tmp/GMOD`, the value should read `/tmp/GMOD:/home/XML-XORT/lib`.

How fix a "rebase" error from Cygwin

If the error message shown in Figure 9.6.24 is encountered when running a Perl code

```
D:\cygwin\bin\cygssl-0.9.7.dll     to     same     address     as
parent(0x1270000) != 0x1280000

15  [main]  perl  3132  sync_with_child:  child  1460(0x660)  died
before initialization with status code 0x1    1019 [main] perl
3132 sync_with_child: *** child state child loading dlls

LOG:   could not receive data from client: Connection reset by
peer

LOG:  unexpected EOF on client connection
```

Figure 9.6.24 The "rebase" error message from Cygwin.

under Cygwin, then try the command `$ rebaseall -v`.

Literature Cited

Altschul, S.F., Gish, W., Miller, W., Myers, E.W., and Lipman, D.J. 1990. Basic local alignment search tool. *J. Mol. Biol.* 5:215:403-10.

Burge, C. and Karlin, S. 1997. Prediction of complete gene structures in human genomic DNA. *J. Mol. Biol.* 268:78-94.

Florea, L., Hartzell, G., Zhang, Z., Rubin, G.M., and Miller, W. 1998. A computer program for aligning a cDNA sequence with a genomic DNA sequence. *Genome Res.* 8:967-974.

Lewis, S.E., Searle, S.M.J., Harris, N., Gibson, M., Iyer, V., Richter, J., Wiel, C., Bayraktarogly, L., Birney, E., Crosby, M.A., Kaminker, J.S., Matthews, B.B., Prochnik, S.E., Smith, C.D., Tupy, J.L., Rubin, G.M., Misra, S., Mungall, C.J., and Clamp, M.E. 2002. Apollo: A sequence annotation editor. *Genome Biol.* 3(12).

Mungall, C.J., Misra, S., Berman, B.P., Carlson, J., Frise, E., Harris, N., Marshall, B., Shu, S., Kaminker, J.S., Prochnik, S.E., Smith, C.D., Smith, E., Tupy, J.L., Wiel, C., Rubin, G.M., and Lewis, S.E. 2002. An integrated computational pipeline and database to support whole-genome sequence annotation. *Genome Biol.* 3(12).

Reese, M.G., Kulp, D., Tammana, H., and Haussler, D. 2000. Genie: Gene finding in *Drosophila melanogaster*. *Genome Res.* 10:529-538.

Stein, L.D., Mungall, C., Shu, S., Caudy, M., Mangone, M., Day, A., Nickerson, E., Stajich, J.E., Harris, T.W., Arva, A., and Lewis, S. 2002. The generic genome browser: A building block for a model organism system database. *Genome Res.* 12:1599-1610.

Zhou, P.L., Letovsky, S., Smutniak, F., Emmert, D., Zhang, P.L., Mungall, C., Cain, S., and Gelbart, W.M 2005. XORT-GameBridge and its application in Apollo driven *Drosophila* genome annotation. In preparation.

Internet Resources

http://www.gmod.org

Web site of GMOD.

http://www.flybase.org

Web site of FlyBase.

http://www.fruitfly.org/annot/gamexml.dtd.txt

Location of GAME XML DTD.

Contributed by Pinglei Zhou,
David Emmert, and Peili Zhang
Harvard University
Cambridge, Massachusetts